SECURE CODING REVIEW REPORT

Internship Task 3 – Secure Coding Review of a Flask Web Application

Submitted By – Lavanya

Domain - Cybersecurity

Project Title

Secure Coding Review and Vulnerability Assessment of a Flask-Based Web Application

ABSTRACT

The purpose of this project is to conduct a Secure Coding Review on a Flask-based web application developed for educational purposes. The application includes user registration, login functionality, a dashboard, and a search feature. During the review process, several security vulnerabilities were identified, including SQL Injection, Cross-Site Scripting (XSS), Plain Text Password Storage, Hardcoded Secret Keys, and Debug Mode Exposure.

The project demonstrates both the vulnerable implementation (app.py) and the remediated implementation (secure_app.py). Security flaws were identified through manual code inspection and testing techniques. Appropriate mitigation measures were implemented to improve the application's overall security posture.

Keywords: Secure Coding, Flask Security, SQL Injection, XSS, Authentication Security, Vulnerability Assessment, Cybersecurity.

INTRODUCTION

Secure coding is the practice of developing software while minimizing security vulnerabilities. Modern web applications process sensitive user information and therefore require strong security controls.

A Secure Coding Review is a systematic examination of application source code to identify weaknesses that could be exploited by attackers. The purpose of this project is to identify security flaws within a Flask-based web application and recommend remediation measures.

This review focuses on authentication mechanisms, input handling, database interactions, session management, and application configuration.

OBJECTIVES

The objectives of this project are:

- Develop a vulnerable Flask application.
- Perform manual code inspection.
- Identify common web vulnerabilities.
- Demonstrate exploitation of identified flaws.
- Perform static code analysis.
- Develop a secure version of the application.
- Compare vulnerable and secure implementations.
- Document findings and recommendations.

SELECTED PROGRAMMING LANUGUAGE

Programming Language: Python

Framework: Flask

Reason for Selection:

Python was selected because it is widely used in web application development, cybersecurity automation, and secure software development. The Flask framework provides a lightweight environment for building web applications and demonstrates common security vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS). This makes it suitable for performing a secure coding review and implementing security improvements.

PROJECT OVERVIEW

The application consists of:

Home Page:
Provides navigation to application modules.

Registration Module:
Allows users to create new accounts.

Login Module:
Authenticates users using stored credentials.

Dashboard:
Accessible after successful login.

Search Module:
Allows user search operations.

Database:
Stores user credentials using SQLite.

The vulnerable version intentionally contains insecure coding practices to demonstrate real-world security issues.

TECHNOLOGY STACK

Programming Language:
Python

Framework:
Flask

Database:
SQLite

Frontend Technologies:
HTML
CSS
JavaScript

Security Analysis Tool:
Bandit

Version Control:
Git and GitHub

Operating System:
Windows

APPLICATION ARCHITECTURE

The application follows a simple client-server architecture.

Components:

Client Browser
↓
Flask Application
↓
SQLite Database

Workflow:

User Registration
→ Database Storage

User Login
→ Authentication

Dashboard Access
→ Session Management

Search Functionality
→ User Input Processing

SECURE CODING REVIEW METHODOLOGY

The review was conducted using the following process:

Phase 1:
Application Development

Phase 2:
Manual Code Inspection

Phase 3:
Vulnerability Discovery

Phase 4:
Exploitation Testing

Phase 5:
Static Analysis

Phase 6:
Remediation

Phase 7:
Validation Testing

Phase 8:
Documentation

VULNERABILITY IDENTIFICATION PROCESS

The following methods were used:

- ✓ Manual Code Review
- ✓ Source Code Inspection
- ✓ Input Validation Testing
- ✓ Authentication Testing
- ✓ Static Analysis Using Bandit
- ✓ Database Inspection
- ✓ Security Configuration Review

SQL INJECTION ANALYSIS

Definition:

SQL Injection occurs when user-controlled input is inserted directly into SQL statements without proper validation.

Affected Module:

Login Functionality

Severity:

HIGH

Risk Rating:

CRITICAL

Evidence:

The vulnerable login query directly concatenated user input into SQL statements.

Example:

SELECT * FROM users
WHERE username='user'
AND password='password'

Impact:

• Authentication Bypass

• Unauthorized Access

• Data Disclosure

• Database Manipulation

Proof of Concept:

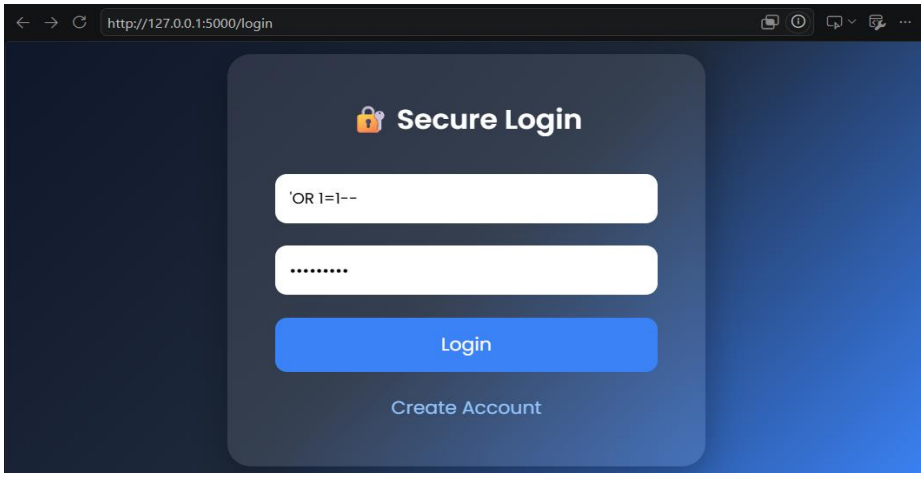
A malicious SQL payload successfully bypassed authentication.

Recommendation:

Use parameterized queries and prepared statements.

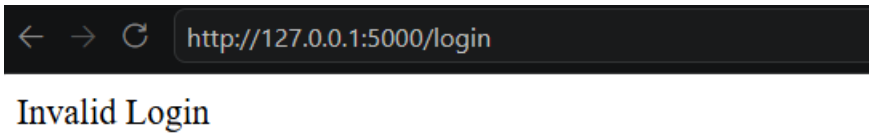
app.py

```
# SQL Injection Vulnerability
query = f"""
SELECT * FROM users
WHERE username='{username}'
AND password='{password}'
"""
```



secure_app.py

```
# Parameterized Query (Prevents SQL Injection)
cursor.execute(
    "SELECT password FROM users WHERE username=?",
    (username,)
)
```



CROSS-SITE SCRIPTING (XSS) ANALYSIS

Definition:

Cross-Site Scripting allows attackers to inject malicious JavaScript into web pages viewed by users.

Affected Module:

Search Functionality

Severity:

HIGH

Risk Rating:

CRITICAL

Impact:

• Session Hijacking

• Cookie Theft

• Website Defacement

• User Redirection

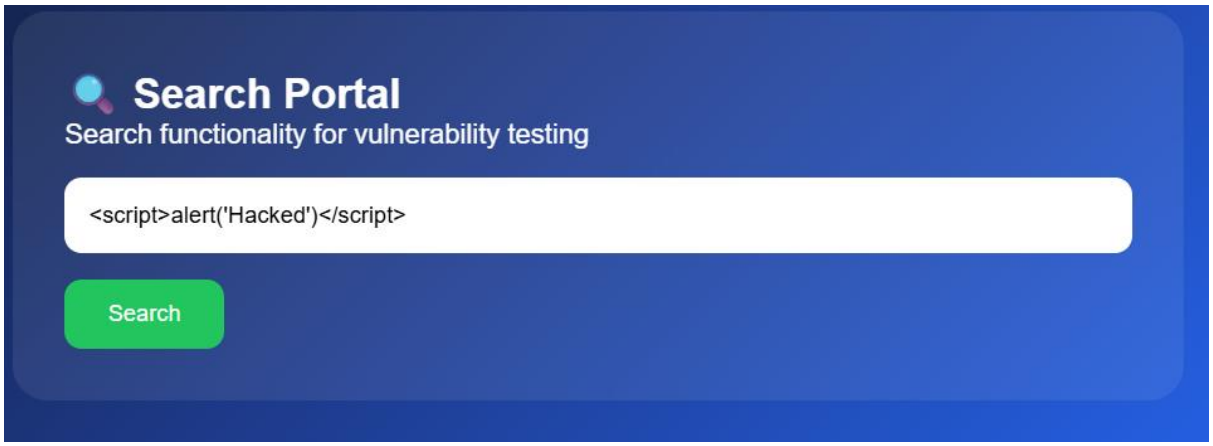
Evidence:

JavaScript payload execution was observed within the vulnerable application.

Recommendation:

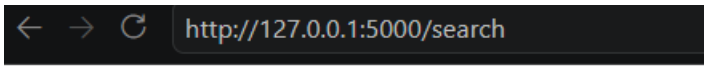
Implement output encoding and input validation.

app.py



secure_app.py

```python
# Secure Search (XSS Protected by Jinja Escaping)
@app.route('/search', methods=['POST'])
def search():

    if 'username' not in session:
        return redirect('/login')

    search_term = request.form['search']

    return render_template(
        'search.html',
        search_term=search_term
    )
```



Search Results

You searched for:

<script>alert('Hacked')</script>

Back

PLAIN TEXT PASSWORD STORAGE ANALYSIS

Definition:

Passwords are stored in readable form without hashing.

Affected Component:

SQLite Database

Severity:

HIGH

Risk Rating:

HIGH

Evidence:

Database records revealed passwords stored as plain text.

Impact:

• Credential Exposure

• Account Compromise

• Password Reuse Attacks

Recommendation:

Use password hashing algorithms.

app.py

[(1, 'admin', '12345'), (2, 'admin', '12345'), (3, 'admin', '12345'), (4, 'admin', 'adimin'), (5, 'admin', 'admin'), (6, 'admin', 'admin'), (7, 'admin', 'admin'), (8, 'admin', 'admin'), (9, 'admin', 'admin'), (10, 'admin',

secure_app.py

'scrypt:32768:8:1$jJhlalXSHELwocuj$43e97e8706addf14793c440a89024be011e62a54f916d19ba76f99c5404d75458f
(12, 'admin',
'scrypt:32768:8:1$TkgyKl3mYa32ITE7$4fff26bfa1b8a2ac0f2ee3ca732c6dec273320d2eb7289a6b53061037d7140545
(13, 'bob',
'scrypt:32768:8:1$bOMA6HXiJEjFmbiF$da8de02918a312e499d8d6b292b563326a961899b030e22b549e26fb0392a8
(14, 'admin',
'scrypt:32768:8:1$a3N66AS7XIGfd9ft$e296617e817487e269dc6218f0ebd70d92f8efa5c6852d279309999bad6e04ce5
(15, 'charles',
'scrypt:32768:8:1$hF9f8KTBpJOAmInP$b9fa5d9a5afe4e1537df0c25a4cb9e426b7c75c9e829a0c7eedb31ed817d8da2

HARDCODED SECRET KEY ANALYSIS

Definition:

Sensitive cryptographic secrets are stored directly in source code.

Severity:

MEDIUM

Risk Rating:

MEDIUM

Evidence:

app.secret_key = "secret123"

Impact:

• Session Manipulation

• Cookie Forgery

• Reduced Security

Recommendation:

Store secrets in environment variables.

app.py

```
app.secret_key = "secret123"   # Hardcoded secret key (vulnerability)
```

secure_app.py

```
# Secure Secret Key
app.secret_key = os.getenv("SECRET_KEY", "change_this_in_production")
```

DEBUG MODE EXPOSURE ANALYSIS

Definition:

Debug mode exposes internal application information.

Severity:

MEDIUM

Risk Rating:

MEDIUM

Evidence:

app.run(debug=True)

Impact:

• Information Disclosure

• Increased Attack Surface

Recommendation:

Disable debug mode in production.

app.py

```
if __name__ == "__main__":
    app.run(debug=True)   # Debug mode enabled (vulnerability)
```

secure_app.py

```
if __name__ == "__main__":
    app.run()
```

STATIC ANALYSIS USING BANDIT

Bandit is a static security analysis tool designed for Python applications.

Purpose:

• Detect insecure coding practices.

• Improve code quality.

• Identify security weaknesses.

Command Used:

bandit -r .

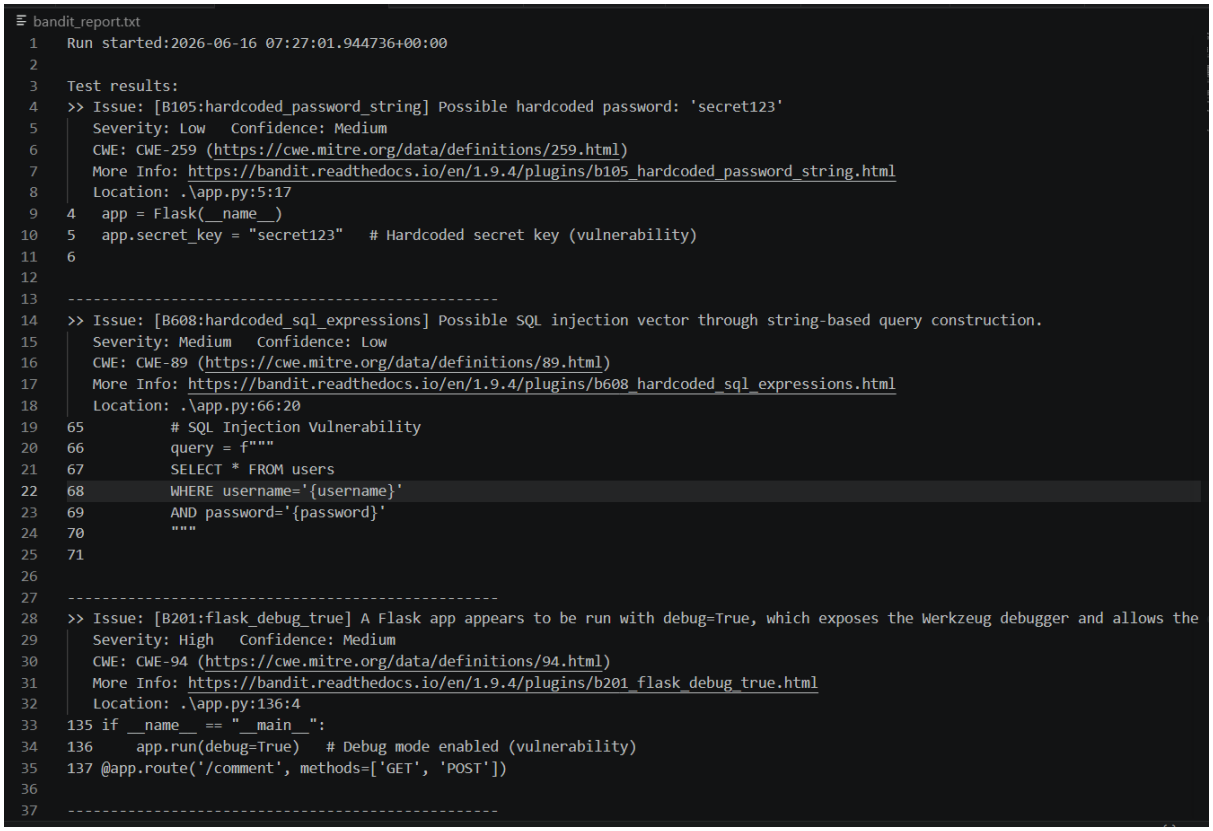
Benefits:

• Automated scanning.

• Faster vulnerability identification.

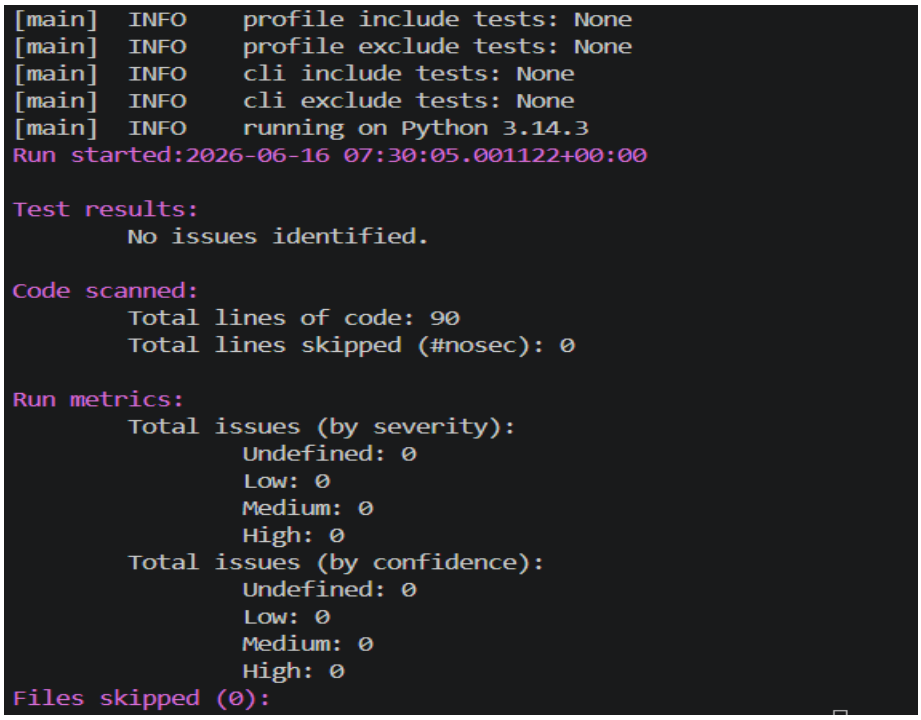
• Improved security awareness.

app.py



```
bandit_report.txt
Run started:2026-06-16 07:27:01.944736+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'secret123'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\app.py:5:17
4   app = Flask(__name__)
5   app.secret_key = "secret123"   # Hardcoded secret key (vulnerability)
6

--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Low
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\app.py:66:20
65          # SQL Injection Vulnerability
66          query = f"""
67          SELECT * FROM users
68          WHERE username='{username}'
69          AND password='{password}'
70          """
71

--------------------------------------------------
>> Issue: [B201:flask_debug_true] A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the
   Severity: High   Confidence: Medium
   CWE: CWE-94 (https://cwe.mitre.org/data/definitions/94.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b201_flask_debug_true.html
   Location: .\app.py:136:4
135 if __name__ == "__main__":
136     app.run(debug=True)   # Debug mode enabled (vulnerability)
137 @app.route('/comment', methods=['GET', 'POST'])
138

--------------------------------------------------
```

secure_app.py



```
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.14.3
Run started:2026-06-16 07:30:05.001122+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 90
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

SECURE IMPLEMENTATION (secure_app.py)

A secure version of the application was developed to mitigate identified vulnerabilities.

Security Improvements:

1. Parameterized Queries
2. Password Hashing
3. Secure Secret Key Handling
4. Session Validation
5. XSS Prevention Through Template Escaping
6. Production-Safe Configuration

The secure version demonstrates industry-standard secure coding practices.

RISK ASSESSMENT TABLE

Vulnerability	**Severity**	**Impact**
SQL Injection	High	Authentication Bypass
Cross-Site Scripting (XSS)	High	Client-Side Code Execution
Plain Text Password Storage	High	Credential Exposure
Hardcoded Secret Key	Medium	Session Security Risk
Debug Mode Enabled	Medium	Information Disclosure

VULNERABILITY COMPARISON

Vulnerability	app.py	secure_app.py
SQL Injection	Present	Fixed
XSS	Present	Fixed
Plain Text Password Storage	Present	Fixed
Hardcoded Secret Key	Present	Fixed
Debug Mode Enabled	Present	Fixed

RECOMMENDATIONS

- Validate all user input.
- Use parameterized SQL queries.
- Store passwords using strong hashing algorithms.
- Secure application secrets.
- Disable debug mode.
- Conduct regular code reviews.
- Implement automated security testing.
- Follow OWASP Secure Coding Guidelines.

CONCLUSION

This project successfully demonstrated the process of conducting a Secure Coding Review on a Flask web application. Multiple vulnerabilities were identified, analyzed, and remediated using secure coding techniques.

The transition from app.py to secure_app.py highlights the importance of secure development practices and proactive vulnerability management. The project reinforces the significance of secure authentication, input validation, proper configuration management, and continuous security testing.